



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Propuesta del Proyecto AnzenCore:

Plataforma de Análisis de Vulnerabilidades Móviles y Calidad de Código

Curso: Calidad y Pruebas de Software

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

Arocutipa Arocutipa, Gian Franco
(2023076790)

Perez Peralta, Fabrizio Salvador Elias **(2023077476)**

Tacna - Perú
2026

CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	GFAA / FSPP	Pendiente de revisión docente	Pendiente	03/07/2026	Versión base de propuesta del proyecto AnzenCore.

Contenido

CONTROL DE VERSIONES	2
RESUMEN EJECUTIVO	4
Resultados esperados principales	4
PROPUESTA NARRATIVA.....	5
1. PLANTEAMIENTO DEL PROBLEMA.....	7
Causas principales	7
Efectos del problema	7
2. JUSTIFICACIÓN DEL PROYECTO.....	8
Justificación técnica.....	8
Justificación académica	8
Justificación económica	8
Justificación social	8
Justificación ambiental.....	8
Justificación metodológica	8
3. OBJETIVO GENERAL.....	8
Objetivos específicos	8
4. BENEFICIOS	9
Beneficios tangibles.....	9
Beneficios intangibles	10
5. ALCANCE	10
Alcance funcional.....	10
Alcance técnico	10
Fuera del alcance	10
6. REQUERIMIENTOS DEL SISTEMA.....	11
Requerimientos no funcionales	12
7. RESTRICCIONES.....	12
8. SUPUESTOS	13
9. RESULTADOS ESPERADOS	13
10. METODOLOGÍA DE IMPLEMENTACIÓN	14

Arquitectura propuesta.....	15
11. ACTORES CLAVES.....	15
12. PAPEL Y RESPONSABILIDADES DEL PERSONAL.....	15
13. PLAN DE MONITOREO Y EVALUACIÓN	16
14. CRONOGRAMA DEL PROYECTO	16
15. HITOS DE ENTREGABLES	17
PRESUPUESTO	18
16. PLANTEAMIENTO DE APLICACIÓN DEL PRESUPUESTO.....	18
17. PRESUPUESTO	18
Escenarios de presupuesto.....	19
18. ANÁLISIS DE FACTIBILIDAD	19
19. EVALUACIÓN FINANCIERA.....	19
Supuestos de evaluación.....	20
20. Resumen técnico ejecutivo.....	20
20.1 Valor diferencial de DataQuest.....	21
20.2 Principios rectores de la propuesta	21
21. Diagnóstico profundo del proyecto DataQuest.....	21
21.1 Inventario técnico real del repositorio	21
21.2 Hallazgos QA relevantes.....	22
21.3 Decisiones técnicas que deben mantenerse	22
22. Análisis estratégico y propuesta de valor ampliada.....	22
22.1 Población objetivo segmentada.....	22
22.2 Propuesta de valor resumida	22
22.3 Alineamiento con objetivos académicos	22
23. EDT/WBS, backlog e historias de usuario.....	23
23.1 Backlog de producto priorizado	23
23.2 Historias de usuario detalladas con criterios de aceptación.....	23
24. Arquitectura técnica, ciclo de datos, API y seguridad.....	24
24.1 Arquitectura lógica enriquecida	24
24.2 Especificación ampliada del endpoint principal.....	25
24.3 Seguridad y privacidad.....	25
24.4 Plan DevOps e infraestructura	25
25. Plan QA extendido y criterios de aceptación.....	26
25.1 Objetivo del plan QA	26
25.2 Matriz de pruebas funcionales principales.....	26
25.3 Criterios de aceptación globales	26
26. Gobierno, monitoreo, comunicación y adopción.....	27

26.1 Matriz RACI extendida	27
26.2 Plan de monitoreo y evaluación enriquecido	27
26.3 Plan de comunicación	28
26.4 Plan de capacitación y adopción	28
27. Presupuesto enriquecido y evaluación financiera académica	29
27.1 Supuestos presupuestales	29
27.2 Presupuesto por escenarios.....	29
27.3 Retorno académico esperado	29
27.4 Evaluación beneficio/costo referencial	30
28. Riesgos, mitigaciones y trazabilidad ampliada	30
28.1 Matriz de riesgos extendida	30
28.2 Trazabilidad problema-objetivo-requerimiento-prueba.....	30

RESUMEN EJECUTIVO

Campo	Descripción
Nombre del proyecto propuesto	AnzenCore: Plataforma de Análisis de Vulnerabilidades Móviles y Calidad de Código.
Propósito del proyecto	Implementar una plataforma que analice APKs, código fuente y repositorios de GitHub para detectar vulnerabilidades OWASP Mobile Top 10 y code smells, generando diagnósticos, métricas de calidad y reportes exportables para apoyar el aprendizaje de calidad y seguridad de software.
Población objetivo	Estudiantes y desarrolladores de aplicaciones móviles Android, docentes de calidad de software, equipos DevSecOps y usuarios que requieren un análisis rápido de vulnerabilidades sin infraestructura compleja.
Monto de inversión referencial	S/ 13,500.00 como presupuesto recomendado académico. Se presenta desglose mínimo, recomendado y extendido.
Duración del proyecto	4 meses, organizado en sprints incrementales de análisis, motor de escaneo, interfaz, API, persistencia, pruebas y documentación.

Resultados esperados principales

- Plataforma web operativa mediante Streamlit para analizar APKs, código fuente y repositorios GitHub, mostrando vulnerabilidades y métricas de calidad.
- Motor de análisis en Python (ApkAnalyzer y FolderAnalyzer) capaz de descomprimir APKs, leer código fuente y detectar patrones de vulnerabilidad mediante expresiones regulares.
- Detección de vulnerabilidades alineadas a OWASP Mobile Top 10 (M1, M5, M7, M8, M9) y CWE, con diagnóstico comprensible para estudiantes.
- API FastAPI disponible para consumo externo mediante los endpoints POST /api/analizar y POST /api/analysis/external/*.
- Persistencia o historial en Supabase/PostgreSQL cuando el entorno esté configurado con las variables de ambiente correspondientes.
- Extensión de Visual Studio Code publicada en el Marketplace y CLI como canales técnicos de integración, según lo evidenciado en el repositorio.
- Plan de QA con pruebas unitarias, integración, interfaz, BDD, API, errores controlados y regresión.
- Documentación técnica y académica alineada a FD03/SRS, FD05/Informe Final y FD06/Propuesta de Proyecto.

Resumen de viabilidad: La propuesta se considera técnica, académica y económicamente viable para un entorno universitario. El proyecto utiliza tecnologías accesibles, separa la lógica del motor de análisis de vulnerabilidades de la interfaz, expone una API para integración externa y permite documentar evidencias de

aprendizaje. Su valor principal no es reemplazar una auditoría de seguridad formal, sino reducir la revisión preliminar manual y mejorar la comprensión del estudiante sobre vulnerabilidades móviles y calidad de código.

PROPUESTA NARRATIVA

AnzenCore se propone como una plataforma educativa y técnica para mejorar el proceso de aprendizaje y validación de la seguridad en aplicaciones móviles y la calidad del código. Su enfoque parte de una dificultad frecuente en cursos de calidad y pruebas de software: comprender y aplicar correctamente el análisis de vulnerabilidades del OWASP Mobile Top 10 y las buenas prácticas de código limpio. Aunque la teoría de seguridad móvil suele explicarse mediante definiciones y ejemplos aislados, los estudiantes enfrentan mayores dificultades cuando deben analizar un APK completo, justificar hallazgos, reconocer patrones de vulnerabilidad y proponer correcciones defendibles.

La propuesta plantea transformar esta revisión en un proceso guiado, repetible y verificable. El usuario podrá subir un archivo APK, una carpeta de código fuente o indicar un repositorio de GitHub desde la interfaz web o desde otros canales técnicos, como API, CLI o extensión de Visual Studio Code. El sistema analizará el artefacto, identificará estructuras relevantes, evaluará posibles vulnerabilidades OWASP Mobile y code smells, y mostrará un diagnóstico orientado al aprendizaje. Este diagnóstico no se limita a marcar errores: también debe explicar el motivo, indicar la severidad y el CWE asociado, y proponer sugerencias de remediación.

Desde el punto de vista técnico, AnzenCore integra un motor core en Python con módulos separados para análisis de APK, análisis de código fuente, escaneo de URLs y consumo de servicios externos de calidad. La interfaz Streamlit facilita el uso académico, mientras que FastAPI permite exponer el motor como servicio HTTP/JSON para integraciones externas. Supabase/PostgreSQL aporta persistencia e historial cuando la configuración del entorno está disponible, y la extensión de VS Code permite acercar el análisis —incluyendo un escaneo de APK 100% local y sin conexión— al editor de código del estudiante.

La propuesta también reconoce límites importantes. AnzenCore no sustituye una auditoría de seguridad formal ni garantiza la detección de toda vulnerabilidad si el APK utiliza ofuscación, cifrado o empaquetado avanzado. En casos complejos, el análisis de seguridad puede requerir herramientas de ingeniería inversa especializadas y criterio experto. Por ello, el sistema se presenta como herramienta académica de apoyo, diagnóstico preliminar y retroalimentación guiada, no como certificador absoluto de la seguridad de una aplicación móvil.

Con esta propuesta, AnzenCore busca generar valor académico mediante una solución concreta, defendible y medible: reducir el tiempo de revisión preliminar de seguridad, mejorar la trazabilidad de las correcciones, apoyar al estudiante en la sustentación de sus hallazgos y promover buenas prácticas de desarrollo seguro de aplicaciones móviles.

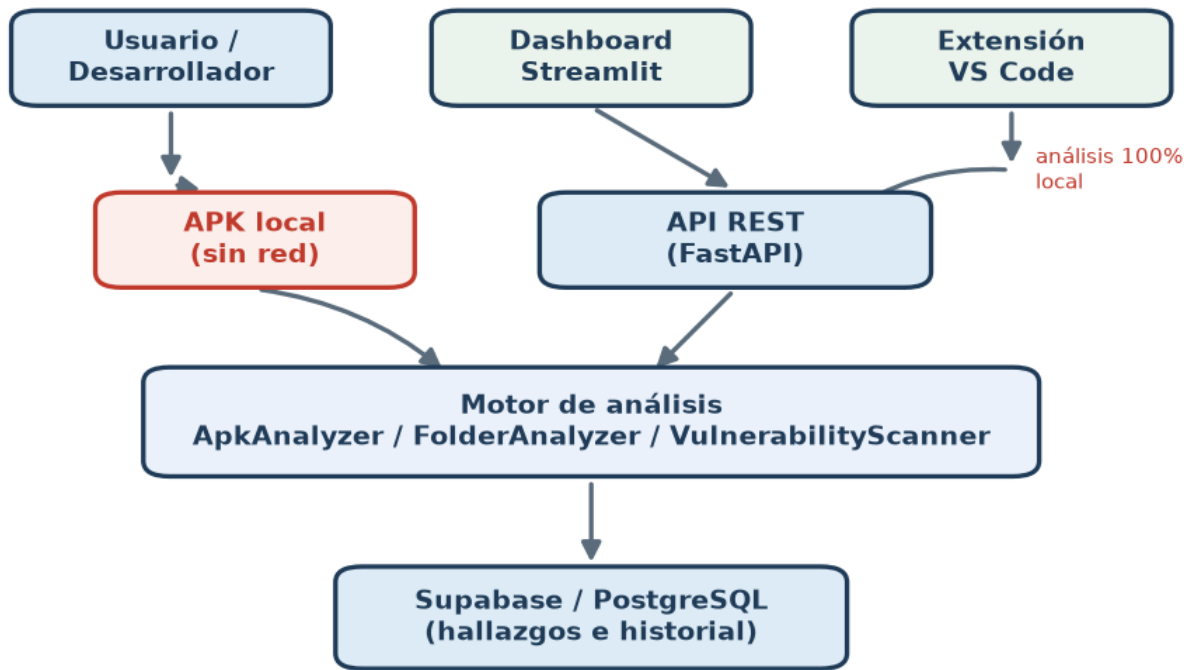


Figura 1. Arquitectura general propuesta de AnzenCore. Fuente: elaboración propia.

1. PLANTEAMIENTO DEL PROBLEMA

Los estudiantes y desarrolladores principiantes presentan dificultades para identificar, justificar y corregir vulnerabilidades de seguridad en aplicaciones móviles Android y en código fuente en general. Esta dificultad se evidencia especialmente cuando deben analizar un APK compilado, interpretar el AndroidManifest, reconocer patrones de código inseguro (secretos hardcodeados, HTTP sin cifrar, criptografía débil, WebView mal configurado), y prevenir la exposición de información sensible.

La revisión manual puede convertirse en una actividad lenta, subjetiva y poco trazable. Un mismo APK puede ser interpretado de forma distinta si no se dispone de criterios claros o de evidencia técnica. Además, cuando el estudiante solo recibe una observación final sin explicación paso a paso, la corrección puede volverse mecánica y no necesariamente mejora la comprensión del diseño seguro de software.

Causas principales

- Desconocimiento práctico del OWASP Mobile Top 10 y de patrones de código inseguro.
- Poca experiencia analizando APKs reales y código descompilado.
- Dificultad para detectar secretos hardcodeados, criptografía débil y comunicación insegura.
- Ausencia de retroalimentación automática y trazable sobre vulnerabilidades.
- Uso frecuente de librerías y prácticas inseguras (HTTP plano, WebView con JavaScript, Random no criptográfico).
- Falta de reportes técnicos que permitan sustentar la corrección del código o del APK.

Efectos del problema

- Aplicaciones móviles con vulnerabilidades explotables y exposición de datos sensibles.
- Correcciones incompletas o justificadas solo de manera verbal.
- Mayor tiempo de revisión por parte del docente.
- Baja comprensión de los riesgos de seguridad móvil y de calidad de código.
- Dificultad para defender el análisis durante una exposición o entrega académica.
- Repetición de vulnerabilidades en proyectos posteriores de desarrollo móvil.

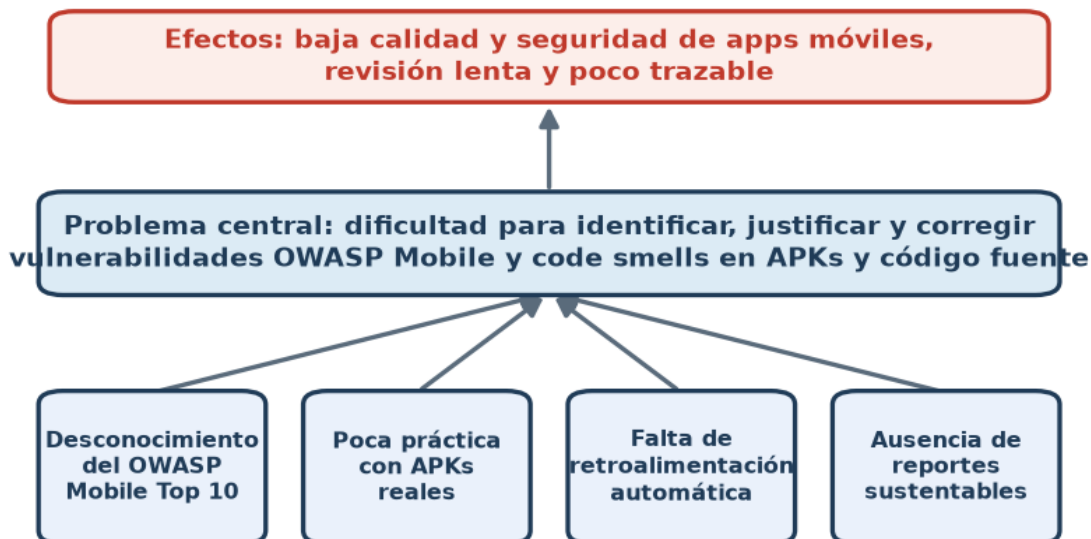


Figura 2. Árbol de problemas de AnzenCore. Fuente: elaboración propia.

2. JUSTIFICACIÓN DEL PROYECTO

Justificación técnica

AnzenCore se justifica técnicamente porque utiliza una arquitectura modular basada en Python, FastAPI, Streamlit y Supabase/PostgreSQL. El repositorio evidencia separación entre app/api (routes, services, schemas) y app/dashboard (controllers, views, services), lo cual facilita mantenimiento, pruebas y evolución. La existencia de Dockerfiles independientes para API y Dashboard, workflows de GitHub Actions, Terraform, CLI y extensión de VS Code fortalece la propuesta técnica al permitir despliegue, automatización e integración con otros entornos.

Justificación académica

El sistema atiende una necesidad recurrente en cursos de calidad y seguridad de software: pasar de la teoría del OWASP Mobile Top 10 a la aplicación práctica sobre APKs y código fuente reales. La herramienta permite experimentar con vulnerabilidades reales, recibir diagnóstico y entender por qué un hallazgo representa un riesgo de seguridad.

Justificación económica

La propuesta es económicamente viable en un entorno académico porque emplea tecnologías libres o de bajo costo. El presupuesto se concentra en desarrollo, documentación, pruebas, diagramación, despliegue controlado y soporte de sustentación.

Justificación social

Al mejorar la formación de estudiantes de ingeniería de sistemas, AnzenCore contribuye a elevar la calidad y seguridad de los proyectos académicos de desarrollo móvil y reduce la brecha entre conocimiento teórico y práctica técnica en seguridad de software.

Justificación ambiental

La propuesta favorece evidencias digitales, reportes electrónicos y revisión automatizada, reduciendo la necesidad de imprimir guías, borradores, observaciones y correcciones manuales.

Justificación metodológica

El sistema permite que la validación sea estructurada, repetible y trazable. Cada revisión puede asociarse a un diagnóstico, una severidad, un CWE/OWASP Mobile y una evidencia de prueba.

3. OBJETIVO GENERAL

Implementar una plataforma académica denominada AnzenCore, orientada a analizar y mejorar la seguridad de aplicaciones móviles Android y la calidad del código fuente mediante la detección de vulnerabilidades del OWASP Mobile Top 10 y code smells, integrando una interfaz web, un motor lógico en Python, una API FastAPI, una extensión de VS Code y mecanismos de persistencia o reporte, con el fin de generar diagnósticos, sugerencias y evidencias que fortalezcan el aprendizaje de la seguridad y calidad de software.

Objetivos específicos

- OE1: Analizar archivos APK subidos por el usuario.
- OE2: Identificar secretos hardcoded, HTTP inseguro y criptografía débil en APKs y código fuente.
- OE3: Detectar configuraciones inseguras de WebView y generadores aleatorios no criptográficos.
- OE4: Detectar direcciones IP hardcodedas y estructura de riesgo del APK (manifest, dex, librerías nativas).
- OE5: Generar diagnósticos comprensibles y orientados al aprendizaje, con severidad, CWE y OWASP Mobile.
- OE6: Proponer sugerencias de remediación con ejemplos de código seguro.

- OE7: Exponer el motor mediante una API FastAPI (/api/analizar y /api/analysis/external/*).
- OE8: Registrar historial o evidencia de análisis cuando la persistencia esté configurada.
- OE9: Validar el sistema mediante pruebas unitarias, funcionales, API, BDD y checklist QA.
- OE10: Documentar la solución para sustentación académica y continuidad técnica, incluyendo la extensión de VS Code publicada en el Marketplace.

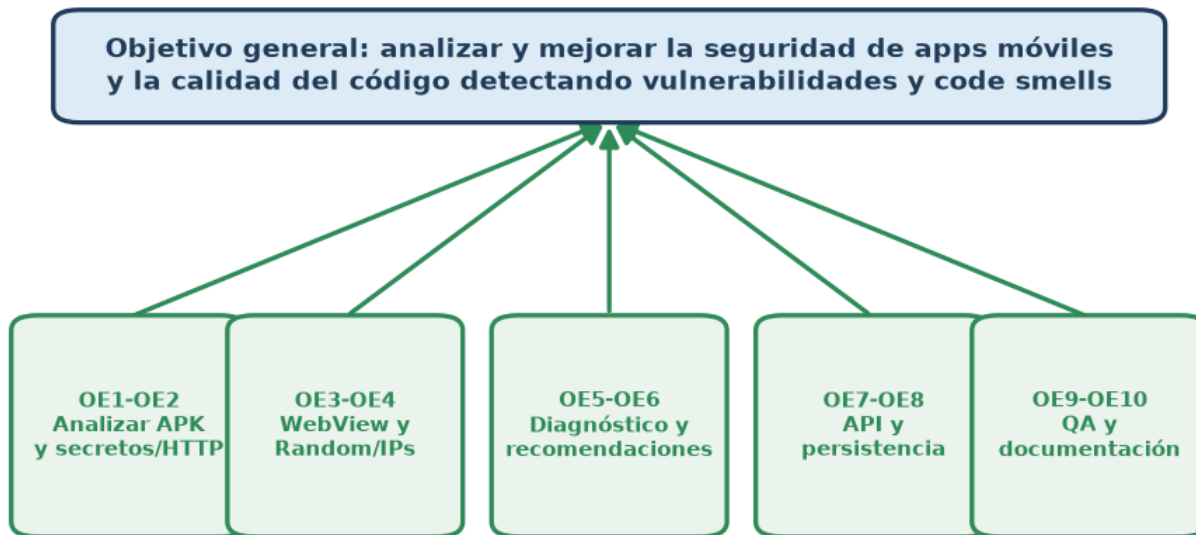


Figura 3. Árbol de objetivos de AnzenCore. Fuente: elaboración propia.

4. BENEFICIOS

Los beneficios de AnzenCore se clasifican en tangibles e intangibles. Los primeros pueden relacionarse con reducción de tiempo, reutilización de la herramienta y centralización de evidencias; los segundos se vinculan con mejora del aprendizaje, seguridad del estudiante al sustentar y fortalecimiento de buenas prácticas de desarrollo seguro.

Concepto	Situación actual	Beneficio con AnzenCore	Impacto esperado
Tiempo de revisión manual	Revisión manual de APKs y código, caso por caso, con observaciones dispersas.	Diagnóstico automatizado de vulnerabilidades y code smells, reutilizable y exportable.	Reducción estimada del 40% al 60% del tiempo de revisión inicial en prácticas académicas.
Evidencia académica	Sustentación basada en explicación verbal o capturas no estandarizadas.	Reporte técnico con severidad, CWE, OWASP Mobile y recomendación de remediación.	Mayor trazabilidad de decisiones y correcciones de seguridad.
Aprendizaje de seguridad móvil	El estudiante memoriza el OWASP Mobile Top 10, pero no siempre lo aplica sobre código real.	Retroalimentación aplicada sobre APKs y código fuente reales, con evidencia de código.	Mejora en comprensión de vulnerabilidades móviles y buenas prácticas.
Reutilización	Cada práctica se analiza manualmente desde cero.	La plataforma (dashboard, API, extensión VS Code) puede usarse en múltiples prácticas y cursos.	Mayor eficiencia docente y estudiantil.
Calidad de proyectos	Apps móviles con secretos hardcodeados, HTTP inseguro o criptografía débil.	Detección automática y recomendaciones de remediación por hallazgo.	Incremento de calidad y seguridad técnica en entregables.

Beneficios tangibles

- Reducción del tiempo de revisión manual de APKs y código fuente.
- Generación de evidencias digitales y reportes reutilizables.
- Estandarización de criterios de diagnóstico preliminar de seguridad.

- Disminución de correcciones repetitivas en prácticas académicas.
- Reutilización del sistema en cursos de calidad de software, seguridad y proyectos integradores.

Beneficios intangibles

- Mayor confianza del estudiante al justificar sus hallazgos de seguridad.
- Fortalecimiento del pensamiento analítico y de seguridad ofensiva/defensiva básica.
- Mejor comprensión del OWASP Mobile Top 10 y de CWE.
- Promoción de buenas prácticas de desarrollo seguro desde etapas tempranas.
- Mayor claridad en la sustentación académica de proyectos.

5. ALCANCE

Alcance funcional

- Ingreso de APK, código fuente, URL o repositorio GitHub por archivo, carpeta o canal técnico.
- Análisis de estructura del APK: manifest, archivos DEX, librerías nativas y bases de datos empaquetadas.
- Detección de secretos hardcodeados, HTTP inseguro, criptografía débil, WebView inseguro, Random inseguro e IPs hardcodeadas.
- Generación de diagnóstico, severidad, CWE, OWASP Mobile y recomendación de remediación.
- Visualización de resultados mediante Streamlit.
- Consumo del motor mediante FastAPI.
- Uso mediante CLI y extensión de Visual Studio Code publicada en el Marketplace, según evidencia del repositorio.
- Historial en Supabase/PostgreSQL si las variables de entorno y tablas están configuradas.
- Pruebas y evidencias para sustentación académica.

Alcance técnico

- Python 3.12+ como lenguaje principal.
- Streamlit como interfaz web (dashboard).
- FastAPI como capa de integración HTTP/JSON.
- Supabase/PostgreSQL para autenticación, hallazgos e historial cuando está configurado.
- Docker (Dockerfile.api y Dockerfile.dashboard) para empaquetado y despliegue controlado.
- GitHub Actions para pruebas, seguridad y despliegue.
- Terraform para infraestructura como código en Azure Container Apps.
- Postman/curl para validación de endpoints.
- Pytest, Behave (BDD) y Mutmut como base de pruebas, con necesidad de fortalecer cobertura.

Fuera del alcance

- No reemplaza una auditoría de seguridad formal ni un pentesting completo.
- No garantiza la detección de toda vulnerabilidad si el APK usa ofuscación o empaquetado avanzado.
- No sustituye el análisis de ingeniería inversa profundo (decompilación de bytecode DEX a Java/Kotlin).
- No corrige automáticamente el código o el APK analizado.
- No debe usarse como única decisión en contextos productivos críticos.
- No debe almacenar ni exponer credenciales sensibles.

6. REQUERIMIENTOS DEL SISTEMA

ID	Descripción	Prioridad	Módulo	Estado	Criterio de aceptación
RF-01	El sistema debe permitir subir un archivo APK para su análisis.	Alta	app/api/routes/analizar.py	Implementado	El usuario envía un APK y recibe un diagnóstico de vulnerabilidades.
RF-02	El sistema debe descomprimir el APK y extraer manifest, archivos DEX y librerías nativas.	Alta	app/dashboard/services/apk_analyzer.py	Implementado	El analizador retorna estructura de archivos y artefactos del APK.
RF-03	El sistema debe detectar secretos hardcodeados (API keys, tokens, JWT) en el código analizado.	Alta	apk_analyzer.py / folder_analyzer.py	Implementado	Se reporta hallazgo "hardcoded_secret" con CWE-798 y evidencia enmascarada.
RF-04	El sistema debe detectar comunicación HTTP insegura (sin cifrar).	Alta	apk_analyzer.py / folder_analyzer.py	Implementado	Se reporta hallazgo con CWE-319 y OWASP M5.
RF-05	El sistema debe detectar uso de criptografía débil (MD5, SHA-1, DES, ECB).	Alta	apk_analyzer.py	Implementado	Se reporta hallazgo "weak_crypto" con CWE-327.
RF-06	El sistema debe detectar configuración insegura de WebView (JavaScript habilitado).	Media	apk_analyzer.py	Implementado	Se reporta hallazgo "insecure_webview" con CWE-79.
RF-07	El sistema debe detectar generadores de números aleatorios inseguros (java.util.Random).	Media	apk_analyzer.py	Implementado	Se reporta hallazgo "insecure_random" con CWE-330.
RF-08	El sistema debe detectar direcciones IP hardcodeadas en el código.	Media	apk_analyzer.py / folder_analyzer.py	Implementado	Se reporta hallazgo "hardcoded_ip" con CWE-200.
RF-09	El sistema debe calcular métricas de calidad de código (LOC, complejidad ciclomática, NOM, NOA).	Alta	folder_analyzer.py	Implementado	El reporte incluye métricas por archivo y totales del proyecto.
RF-10	El sistema debe analizar repositorios públicos de GitHub sin requerir autenticación.	Alta	app/api/routes/external_analysis.py	Implementado	POST /api/analysis/external/github responde con métricas y code smells.
RF-11	El sistema debe permitir analizar una carpeta local completa mediante carga de archivos.	Alta	app/api/services/folder_analyzer.py	Implementado	POST /api/analysis/external/upload_folder responde por archivo.
RF-12	El sistema debe exponer una API REST documentada con FastAPI/OpenAPI.	Alta	app/api/main.py	Implementado	/docs muestra los endpoints disponibles.
RF-13	El sistema debe presentar resultados mediante un dashboard interactivo en Streamlit.	Alta	app/dashboard/views	Implementado	El usuario visualiza hallazgos, métricas y gráficos tras el análisis.
RF-14	El sistema debe permitir exportar reportes de resultados.	Media	report_export_service.py	Implementado	El usuario descarga un reporte con los hallazgos detectados.
RF-15	El sistema debe registrar hallazgos y reportes en una base de datos gestionada.	Media	Supabase/PostgreSQL	Implementado / requiere configuración	Los hallazgos persisten cuando SUPABASE_URL/SUPABASE_KEY están configurados.
RF-16	El sistema debe incluir autenticación de usuarios en el dashboard.	Media	app/api/routes/auth.py	Implementado	Login/registro dependen de Supabase configurado.
RF-17	El sistema debe permitir el análisis de vulnerabilidades desde Visual Studio Code.	Media	vscode-extension/	Implementado	La extensión analiza workspace, carpetas, archivos y APKs desde el editor.
RF-18	El análisis de APK desde la extensión de VS Code debe funcionar sin conexión al servidor.	Alta	vscode-extension/src/apkAnalyzer.ts	Implementado	El escaneo de APK corre localmente con fflate y regex, sin llamadas HTTP.
RF-19	El sistema debe permitir ejecución	Media	Dockerfile.api / Dockerfile.dashboard	Implementado	Cada imagen construye y ejecuta el servicio correspondiente.

AnzenCore - FD06 / Propuesta de Proyecto

	mediante contenedores Docker independientes para API y Dashboard.				
RF-20	El sistema debe contar con documentación de instalación, API e integración externa.	Alta	README.md / API_INTEGRATION.md	Implementado	Documentación describe instalación, endpoints y ejemplos de consumo.

Requerimientos funcionales

Requerimientos no funcionales

ID	Categoría	Requerimiento no funcional	Prioridad	Criterio de aceptación
RNF-01	Usabilidad	El dashboard debe presentar hallazgos con severidad y recomendación en lenguaje comprensible.	Alta	El usuario entiende el hallazgo y la remediación sin revisar código fuente.
RNF-02	Rendimiento	El análisis local de un APK desde la extensión VS Code debe completarse en menos de 2 segundos para archivos típicos.	Media	Medición sobre assets/app-debug.apk (~12MB) completada en menos de 2 segundos.
RNF-03	Seguridad	No deben exponerse credenciales de Supabase, tokens ni secretos en el repositorio.	Crítica	Variables sensibles solo en .env/.env.example, sin secretos versionados.
RNF-04	Mantenibilidad	El código debe mantenerse separado por capas: api, dashboard, services, routes, schemas.	Alta	Cada responsabilidad tiene carpeta y módulo claro.
RNF-05	Portabilidad	Debe ejecutarse en Windows/Linux con Python 3.12+ y Node.js 18+ (extensión).	Media	Instalación reproducible mediante requirements.txt y package.json.
RNF-06	Interoperabilidad	La API debe responder JSON estándar y documentarse con OpenAPI/FastAPI docs.	Alta	/docs disponible al ejecutar la API.
RNF-07	Trazabilidad	Cada hallazgo debe vincularse a severidad, CWE, OWASP Mobile y recomendación.	Alta	El reporte muestra estos campos en cada vulnerabilidad.
RNF-08	Calidad académica	El sistema debe diferenciar diagnóstico automatizado de una auditoría de seguridad formal.	Alta	Documentación explica alcance y limitaciones del análisis estático.
RNF-09	Pruebas	Debe existir suite de pruebas unitarias, de integración, interfaz y BDD.	Alta	Pytest y Behave ejecutan pruebas sobre servicios de análisis y rutas API.
RNF-10	Escalabilidad futura	Debe permitir agregar nuevos detectores de vulnerabilidades u otras plataformas (iOS).	Media	Arquitectura de servicios permite nuevos analizadores sin acoplamiento.
RNF-11	Disponibilidad	El análisis local (APK en VS Code, code smells básicos) debe funcionar aunque Supabase no esté configurado.	Media	El escaneo de APK no depende de conexión externa.
RNF-12	Documentación	README, API_INTEGRATION.md y documentación de la extensión deben estar actualizados.	Alta	Documentos describen instalación, endpoints e integración.

7. RESTRICCIONES

- La precisión del diagnóstico depende de la calidad del APK/código analizado y de la disponibilidad de patrones reconocibles.

- Algunas vulnerabilidades requieren criterio humano o herramientas de ingeniería inversa para confirmar falsos positivos.
- El motor actual utiliza detección basada en expresiones regulares, sin decompilación completa de bytecode DEX.
- La API requiere entorno Python y dependencias instaladas.
- Supabase requiere conexión a internet si se usa como servicio externo.
- Docker requiere instalación previa y permisos del sistema operativo.
- La extensión de VS Code requiere Node.js 18+ y compilación TypeScript (esbuild) para el análisis de código vía API.
- El sistema no debe exponer claves, tokens ni credenciales.
- El alcance es académico y no certificador formal de seguridad.

8. SUPUESTOS

- El usuario subirá un APK válido o código fuente razonablemente estructurado.
- El docente permitirá el uso de evidencias digitales como parte de la sustentación.
- El entorno local contará con Python 3.12+ y Node.js 18+ instalados.
- El equipo tendrá acceso al repositorio y a las dependencias del proyecto.
- Las pruebas se realizarán con APKs y código con vulnerabilidades conocidas y casos límite.
- La documentación se mantendrá actualizada durante el desarrollo.
- El proyecto se ejecutará primero en un entorno académico/local antes de un despliegue público.
- El uso de Supabase se realizará con claves configuradas en variables de entorno.

9. RESULTADOS ESPERADOS

ID	Resultado esperado	Criterio de cumplimiento	Evidencia
RE-01	API REST funcional	Usuario puede analizar APK, código fuente, URL o repo GitHub vía POST /api/analizar.	Captura de Swagger /docs y prueba con Postman/curl.
RE-02	Motor de detección de vulnerabilidades operativo	ApkAnalyzer y FolderAnalyzer retornan hallazgos con severidad, CWE y OWASP Mobile.	Pruebas unitarias y reporte JSON.
RE-03	Dashboard Streamlit funcional	Usuario visualiza métricas, hallazgos y gráficos tras el análisis.	Captura de pantalla / video demo.
RE-04	Persistencia de hallazgos	Resultados pueden guardarse en Supabase/PostgreSQL si está configurado.	Registro en tablas gestionadas de Supabase.
RE-05	Extensión de VS Code publicada	Extensión disponible en el Marketplace con análisis de código y APK.	Ficha del Marketplace y .vsix verificado.
RE-06	Plan QA ejecutable	Casos de prueba unitarios, integración, interfaz y BDD definidos y ejecutados.	Reporte pytest/behave y matriz QA.
RE-07	Documentación final	FD06, manual, anexos y checklist listos.	Documento final revisado.

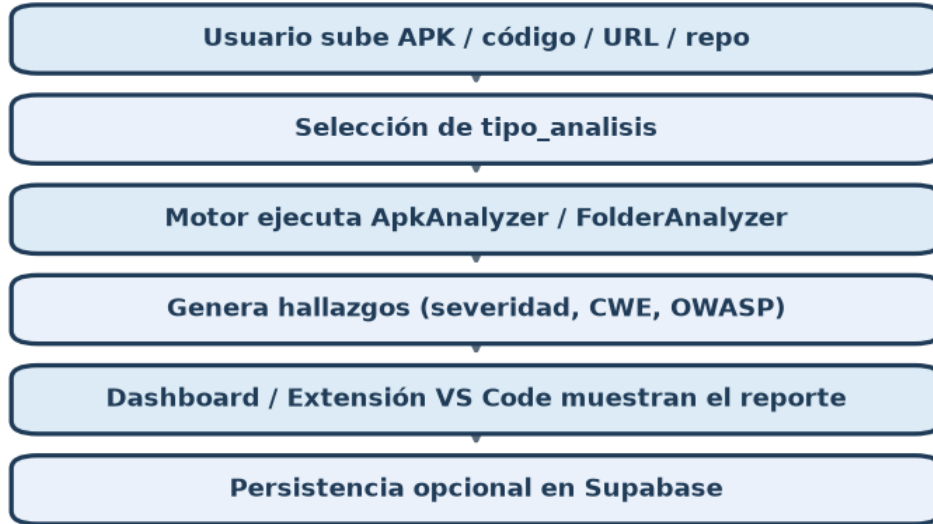


Figura 4. Flujo funcional del proceso de análisis. Fuente: elaboración propia.

10. METODOLOGÍA DE IMPLEMENTACIÓN

La implementación se plantea mediante un enfoque incremental basado en SCRUM académico. Cada sprint genera un entregable verificable, permitiendo validar el avance de forma continua y reducir riesgos de integración. El proyecto se organiza en 4 meses, con énfasis en análisis, motor de detección de vulnerabilidades, interfaz, API, extensión VS Code, persistencia, pruebas, documentación y sustentación.

Sprint	Duración	Objetivo	Actividades	Entregable	Criterio de aceptación
Sprint 0	1 semana	Preparación	Revisión de FD03, FD05, repositorio, alcance, backlog y entorno.	Backlog y entorno base	Alcance aprobado.
Sprint 1	2 semanas	Análisis y requisitos	Formalizar problema, actores, RF/RNF y criterios de aceptación.	Requerimientos consolidados	RF y RNF revisados.
Sprint 2	2 semanas	Motor de análisis de APK	Implementar detección de secretos, HTTP inseguro, criptografía débil y estructura del APK.	ApkAnalyzer funcional	Detecta hallazgos sobre APK de prueba.
Sprint 3	2 semanas	Motor de code smells	Fortalecer FolderAnalyzer: LOC, complejidad, NOM, NOA y smells.	FolderAnalyzer funcional	Casos de prueba pasan.
Sprint 4	2 semanas	API REST y dashboard	Exponer endpoints /api/analizar y /api/analysis/external/*, mejorar vistas Streamlit.	API y dashboard funcionales	Usuario ejecuta flujo completo.
Sprint 5	2 semanas	Persistencia	Configurar Supabase/PostgreSQL, autenticación e historial de reportes.	BD funcional	Reportes se registran correctamente.
Sprint 6	2 semanas	Extensión VS Code	Implementar comandos de análisis de workspace, carpeta y APK local.	Extensión funcional	Los 4 comandos operan correctamente.
Sprint 7	2 semanas	DevOps y QA	Docker, CI/CD, Terraform, pruebas y regresión.	Pipeline y QA	Workflows ejecutan pruebas reales.
Sprint 8	1 semana	Cierre	Documentación, evidencias, sustentación, publicación en Marketplace.	FD06 final y extensión publicada	Entrega lista para revisión.

Arquitectura propuesta

La arquitectura propuesta separa la interfaz de usuario, el motor core, la API, la persistencia y la automatización. Esta separación permite que el motor de análisis de vulnerabilidades pueda ser reutilizado desde Streamlit, FastAPI, CLI, la extensión de VS Code o futuros agentes IA. El componente de persistencia queda desacoplado para permitir que el análisis local (incluido el escaneo de APK 100% offline en la extensión de VS Code) funcione aun cuando Supabase no esté disponible.

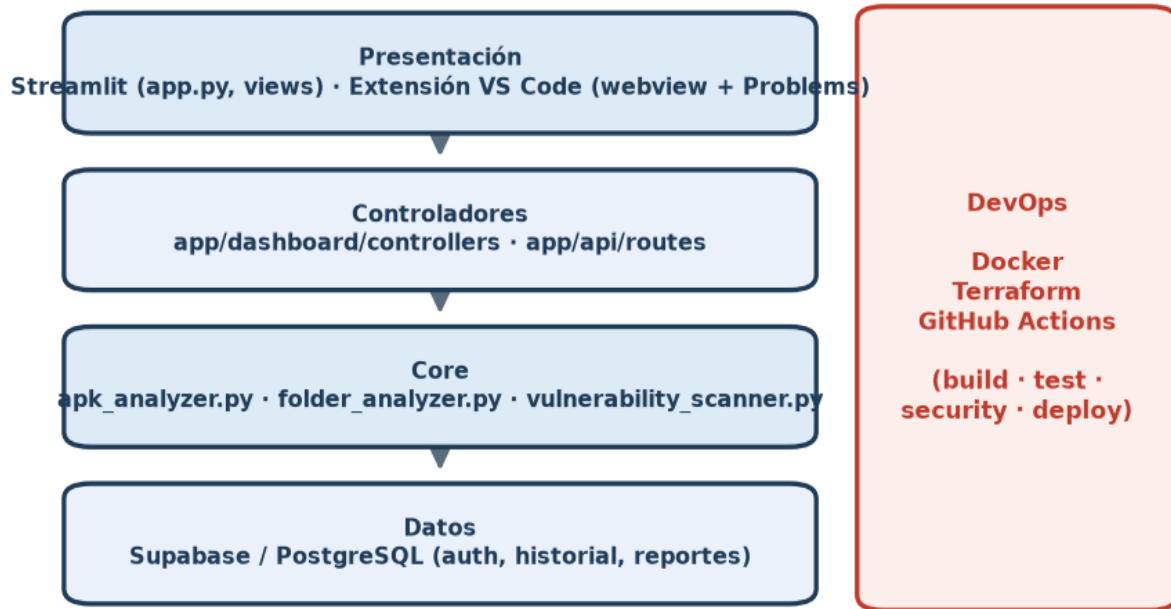


Figura 5. Arquitectura técnica ampliada de AnzenCore. Fuente: elaboración propia.

11. ACTORES CLAVES

Actor / cargo	Responsabilidad	Participación	Entregable asociado
Estudiante desarrollador	Construye, integra, documenta y sustenta el proyecto.	Alta	Código, pruebas, FD06 y demo.
Docente evaluador	Valida alcance, coherencia académica y calidad del entregable.	Alta	Observaciones y aprobación.
Usuario desarrollador móvil	Usa la plataforma para analizar APKs y aprender sobre vulnerabilidades OWASP Mobile.	Media	Feedback de uso y capturas.
Usuario docente de calidad	Puede usar reportes como apoyo en revisión preliminar de seguridad.	Media	Criterios de evaluación.
Administrador técnico	Configura entorno, variables y despliegue en Azure.	Media	Instalación y configuración.
API consumer	Consume endpoints de análisis desde otro cliente o CI/CD.	Media	Prueba Postman/curl.
QA tester	Ejecuta pruebas funcionales, API, regresión y seguridad básica.	Alta	Matriz QA y evidencias.
Revisor académico	Evalúa consistencia de documentos FD03, FD05 y FD06.	Media	Checklist de documentación.

12. PAPEL Y RESPONSABILIDADES DEL PERSONAL

Actividad	Responsable	RACI	Responsabilidad	Evidencia
Análisis funcional	Estudiante desarrollador	R/A	Levantar problema, alcance, beneficios y restricciones.	FD06 revisado.
Desarrollo motor de análisis	Desarrollador Python	R	Implementar ApkAnalyzer y FolderAnalyzer: secretos,	Pruebas unitarias.

AnzenCore - FD06 / Propuesta de Proyecto

			HTTP, criptografía, WebView, IPs, LOC y complejidad.	
Desarrollo dashboard	Desarrollador Streamlit	R	Construir flujo de entrada y visualización de resultados.	Capturas UI.
Desarrollo API	Backend developer	R	Exponer endpoint, validar entradas y errores.	Prueba Postman.
Base de datos	Administrador técnico	R/C	Configurar Supabase, autenticación e historial.	Esquema de base de datos.
QA	Analista QA	R/A	Diseñar y ejecutar matriz de pruebas.	Reporte QA.
Documentación	Redactor técnico	R	Actualizar FD03, FD05, FD06, manual y anexos.	Documentos finales.
Sustentación	Estudiante / expositor	A	Preparar demo, discurso y evidencias.	Presentación oral.
Mantenimiento	Equipo técnico	C/I	Corregir errores y ampliar funcionalidades.	Backlog de mejoras.

13. PLAN DE MONITOREO Y EVALUACIÓN

Indicador	Fórmula	Meta	Frecuencia	Responsable	Evidencia
Porcentaje de RF implementados	RF implementados / RF totales x 100	>= 75% para entrega académica	Semanal	Estudiante	Matriz RF
Pruebas ejecutadas	Casos ejecutados / casos planificados x 100	>= 80%	Por sprint	QA	Reporte Pytest/Behave
Pruebas aprobadas	Casos aprobados / casos ejecutados x 100	>= 85%	Por sprint	QA	Matriz QA
Tiempo promedio de análisis	Promedio de segundos por APK/código	< 3 s en análisis local de VS Code	Quincenal	Desarrollador	Medición local
Errores controlados	Errores con mensaje claro / errores totales	>= 90%	Quincenal	Backend	Logs/API
Cobertura documental	Secciones completas / secciones esperadas	100%	Cierre	Documentador	Checklist
Evidencias generadas	Capturas y reportes disponibles	>= 10 evidencias	Cierre	Estudiante	Anexo 05
Satisfacción académica	Encuesta breve 1 a 5	>= 4	Final	Docente/usuarios	Encuesta

14. CRONOGRAMA DEL PROYECTO

Actividad	Mes 1	Mes 2	Mes 3	Mes 4	Responsable	Entregable	Estado
Análisis y levantamiento	X				Estudiante	Alcance y problema	Planificado
FD03/SRS y arquitectura	X	X			Estudiante	Requerimientos y diagramas	Realizado/ajustable
Motor de análisis de APK		X			Desarrollador	ApkAnalyzer funcional	Implementado/mejorable
Motor de code smells		X	X		Desarrollador	FolderAnalyzer funcional	Implementado/mejorable
Dashboard Streamlit		X	X		UI	Pantallas funcionales	Implementado/parcial
API FastAPI			X		Backend	Endpoints probados	Implementado
Supabase/PostgreSQL			X		BD	Historial y auth	Planificado/configurable
Extensión VS Code			X		Integración	Extensión publicada	Implementado
Docker/CI/Terraform			X		DevOps	Pipeline	Implementado técnico
Pruebas y correcciones			X	X	QA	Matriz QA	Parcial/recomendado
FD05/FD06 y anexos				X	Documentador	Documentos finales	En elaboración
Sustentación				X	Estudiante	Demo final	Planificado

AnzenCore - FD06 / Propuesta de Proyecto

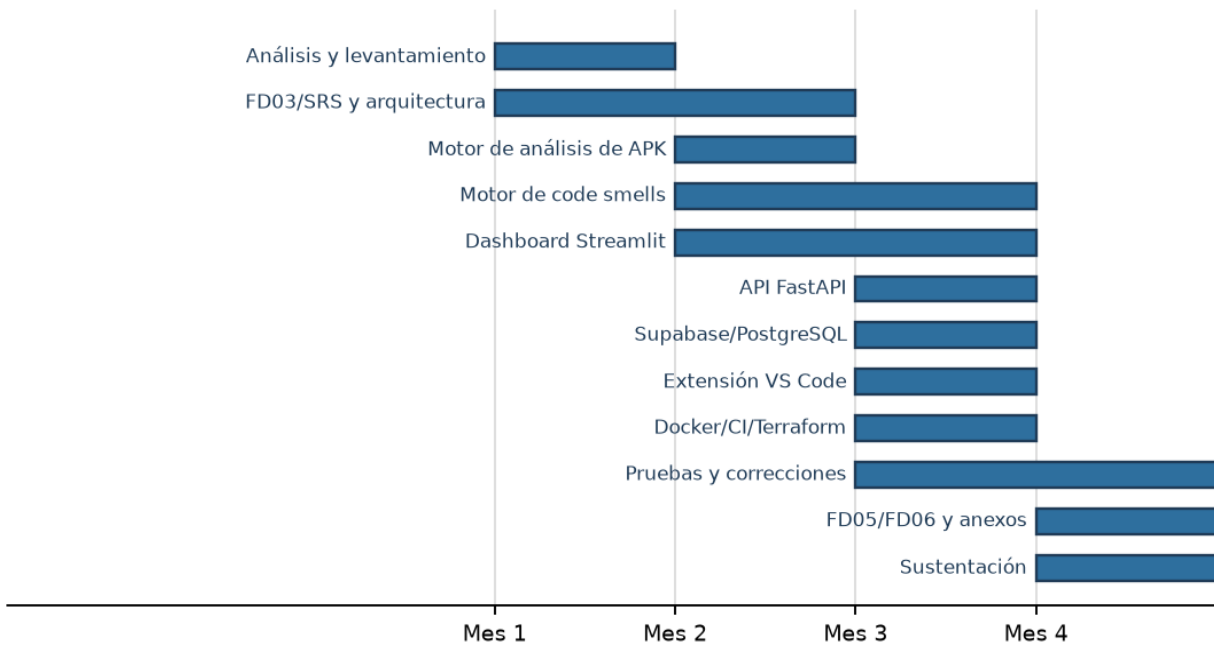


Figura 6. Cronograma tipo Gantt del proyecto AnzenCore. Fuente: elaboración propia.

15. HITOS DE ENTREGABLES

Hito	Fecha estimada	Entregable	Criterio de aceptación	Responsable
H1	Semana 1	Alcance aprobado	Problema, objetivo y alcance validados.	Estudiante
H2	Semana 3	Motor de APK inicial	ApkAnalyzer detecta estructura y hallazgos básicos.	Desarrollador
H3	Semana 5	Motor de code smells	FolderAnalyzer calcula métricas y detecta smells.	Desarrollador
H4	Semana 7	Dashboard funcional	Usuario ejecuta flujo desde Streamlit.	UI
H5	Semana 9	API funcional	POST /api/analizar responde correctamente.	Backend
H6	Semana 11	Persistencia/historial	Supabase registra hallazgos e historial.	BD
H7	Semana 13	Extensión VS Code publicada	Extensión disponible en el Marketplace con análisis de APK local.	Desarrollador
H8	Semana 15	Pruebas QA y documentación final	Matriz QA ejecutada y FD06 completo.	QA/Documentador
H9	Semana 16	Sustentación	Demo y exposición listas.	Estudiante

PRESUPUESTO

16. PLANTEAMIENTO DE APLICACIÓN DEL PRESUPUESTO

El presupuesto de AnzenCore se plantea como una inversión académica referencial. Su finalidad es estimar el esfuerzo técnico y documental requerido para convertir el proyecto en una solución sustentable, probada y defendible. No representa una propuesta comercial oficial; los montos se expresan en soles y se basan en horas estimadas de desarrollo, documentación, pruebas y soporte de sustentación.

- Desarrollo del motor core: análisis de APK, análisis de código fuente y detección de vulnerabilidades.
- Desarrollo de interfaz Streamlit y mejora de experiencia de usuario.
- Desarrollo y validación de API FastAPI.
- Configuración de persistencia con Supabase/PostgreSQL.
- Pruebas unitarias, API, interfaz, regresión y seguridad básica.
- Documentación FD06, diagramas, manuales y anexo único consolidado.
- Despliegue controlado con Docker, Terraform y automatización CI/CD.
- Desarrollo y publicación de la extensión de Visual Studio Code en el Marketplace.

Presupuesto recomendado: S/ 13,500

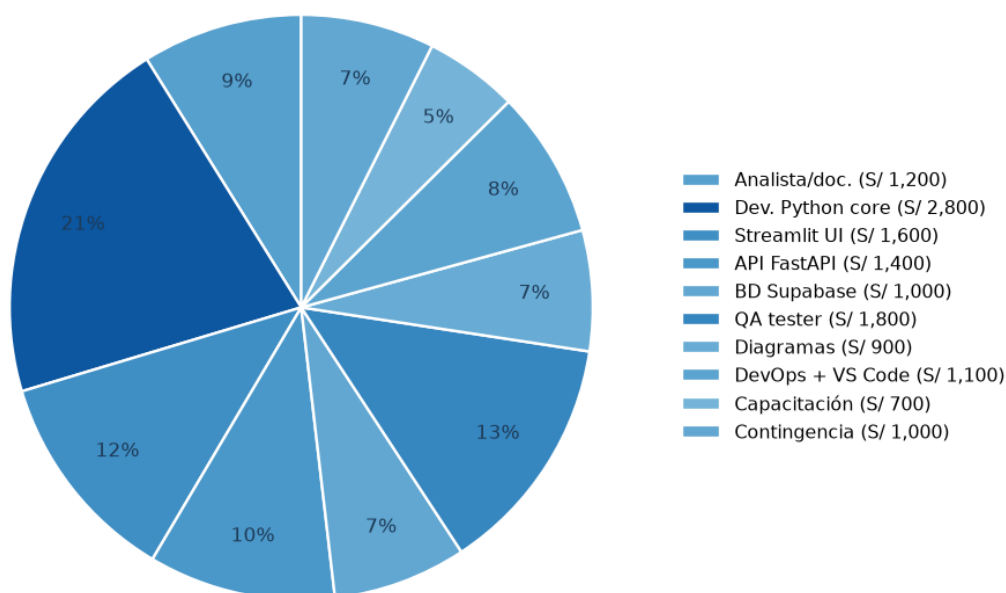


Figura 7. Distribución referencial del presupuesto recomendado. Fuente: elaboración propia.

17. PRESUPUESTO

Concepto	Cantidad	Costo unitario	Subtotal	Observación
Analista funcional / documentación	1	S/ 1,200.00	S/ 1,200.00	Levantamiento, alcance, FD06 y trazabilidad.
Desarrollador Python core	1	S/ 2,800.00	S/ 2,800.00	Motor de análisis de APK y code smells.
Desarrollador Streamlit UI	1	S/ 1,600.00	S/ 1,600.00	Pantallas, flujo de usuario y visualización.
Desarrollador API FastAPI	1	S/ 1,400.00	S/ 1,400.00	Endpoints, manejo de errores y documentación OpenAPI.

AnzenCore - FD06 / Propuesta de Proyecto

Base de datos Supabase/PostgreSQL	1	S/ 1,000.00	S/ 1,000.00	Historial, autenticación y conexión.
QA tester	1	S/ 1,800.00	S/ 1,800.00	Matriz de pruebas, regresión y evidencias.
Diseño de diagramas y evidencias	1	S/ 900.00	S/ 900.00	Diagramas, capturas y anexos.
DevOps académico y extensión VS Code	1	S/ 1,100.00	S/ 1,100.00	Docker, Terraform, GitHub Actions y publicación de la extensión en el Marketplace.
Capacitación / sustentación	1	S/ 700.00	S/ 700.00	Guion de demo y preparación de exposición.
Contingencia técnica	1	S/ 1,000.00	S/ 1,000.00	Correcciones no previstas.
TOTAL RECOMENDADO			S/ 13,500.00	Presupuesto académico referencial.

Escenarios de presupuesto

Escenario	Monto	Incluye	Uso recomendado
Mínimo	S/ 8,500.00	Documento, motor básico (APK + code smells), Streamlit y API básica.	Adecuado para entrega académica reducida.
Recomendado	S/ 13,500.00	Motor completo, UI, API, Supabase, extensión VS Code, QA, diagramas y documentación completa.	Mejor equilibrio entre calidad y costo.
Extendido	S/ 18,500.00	Incluye mejora avanzada de la extensión VS Code, Swagger detallado, mayor cobertura QA y despliegue cloud en Azure.	Recomendado para producto demostrable más sólido.

18. ANÁLISIS DE FACTIBILIDAD

Tipo	Evaluación	Argumento	Riesgo	Mitigación
Técnica	Alta	Las tecnologías del proyecto son accesibles y ya están presentes en el repositorio: Python, Streamlit, FastAPI, Supabase, Docker, Terraform, GitHub Actions y extensión de VS Code.	Dependencias o entorno mal configurado.	Documentar instalación y usar .env.example.
Económica	Viable	El proyecto usa herramientas libres o de bajo costo; el mayor costo corresponde a esfuerzo humano.	Sobrecosto por ampliación de alcance.	Controlar backlog y priorizar MVP.
Operativa	Alta	Estudiantes y docentes pueden usar la interfaz web o la extensión de VS Code sin aprender herramientas complejas.	Curva de aprendizaje inicial.	Manual breve y ejemplos de APK/código.
Académica	Alta	Atiende una necesidad clara de cursos de calidad y seguridad de software y mejora trazabilidad de correcciones.	Dependencia excesiva de la herramienta.	Aclarar que es apoyo, no reemplazo de auditoría formal.
Legal	Alta	No requiere datos personales reales si se usa en modo académico con APKs de prueba.	Credenciales expuestas.	Uso de variables de entorno y revisión de secretos.
Ambiental	Alta	Reduce impresión de guías y correcciones mediante reportes digitales.	Uso de recursos cloud innecesarios.	Usar el análisis local (VS Code) para prácticas.
Mantenimiento	Media-alta	La arquitectura modular permite extender detectores y pruebas.	Pruebas insuficientes.	Ampliar Pytest y casos de regresión.

19. EVALUACIÓN FINANCIERA

La evaluación financiera de AnzenCore se presenta como una estimación académica referencial. Dado que el proyecto no se plantea como producto comercial inmediato, el retorno se mide por ahorro de tiempo, reutilización en cursos, mejora de calidad académica y reducción de reprocesos de seguridad. Para fines de propuesta, se consideran tres escenarios de valor.

Escenario	Inversión	Beneficio esperado	Retorno estimado	Relación beneficio/costo
Conservador	S/ 8,500.00	Uso en un curso y 25 estudiantes. Ahorro de 15	Retorno académico en un ciclo.	B/C cualitativo positivo.

		minutos por revisión preliminar de seguridad.		
Moderado	S/ 13,500.00	Uso en varios cursos, 60 estudiantes y reutilización de la extensión VS Code y reportes.	Retorno académico en 2 ciclos.	B/C referencial 1.25.
Optimista	S/ 18,500.00	Uso institucional, API, extensión VS Code con mayor adopción y mayor cobertura QA.	Retorno académico y técnico en 2 a 3 ciclos.	B/C referencial 1.45.

Supuestos de evaluación

- El valor económico se interpreta como ahorro de tiempo y mejora de calidad académica, no como ingreso comercial directo.
- La herramienta puede reutilizarse en múltiples prácticas, cursos y semestres.
- El costo de infraestructura se mantiene bajo usando entornos gratuitos, locales o el escaneo 100% local de la extensión VS Code.
- El retorno aumenta si el sistema se usa como apoyo docente o repositorio de evidencias de seguridad.

20. Resumen técnico ejecutivo

AnzenCore se propone como una solución académica de análisis de vulnerabilidades móviles y calidad de código orientada a estudiantes, docentes y desarrolladores principiantes. A diferencia de una revisión manual tradicional, la propuesta busca que el usuario suba un APK o código fuente, reciba un diagnóstico por tipo de vulnerabilidad y obtenga recomendaciones comprensibles para mejorar la seguridad del proyecto.

El enfoque del proyecto no consiste únicamente en detectar errores, sino en convertir el proceso de análisis de seguridad en una experiencia formativa. Por ello, la propuesta combina interfaz web, motor lógico, API, extensión de editor, repositorio documentado, pruebas y evidencias. El valor principal es pedagógico: reducir la brecha entre teoría de seguridad móvil (OWASP Mobile Top 10) y aplicación práctica sobre APKs reales.

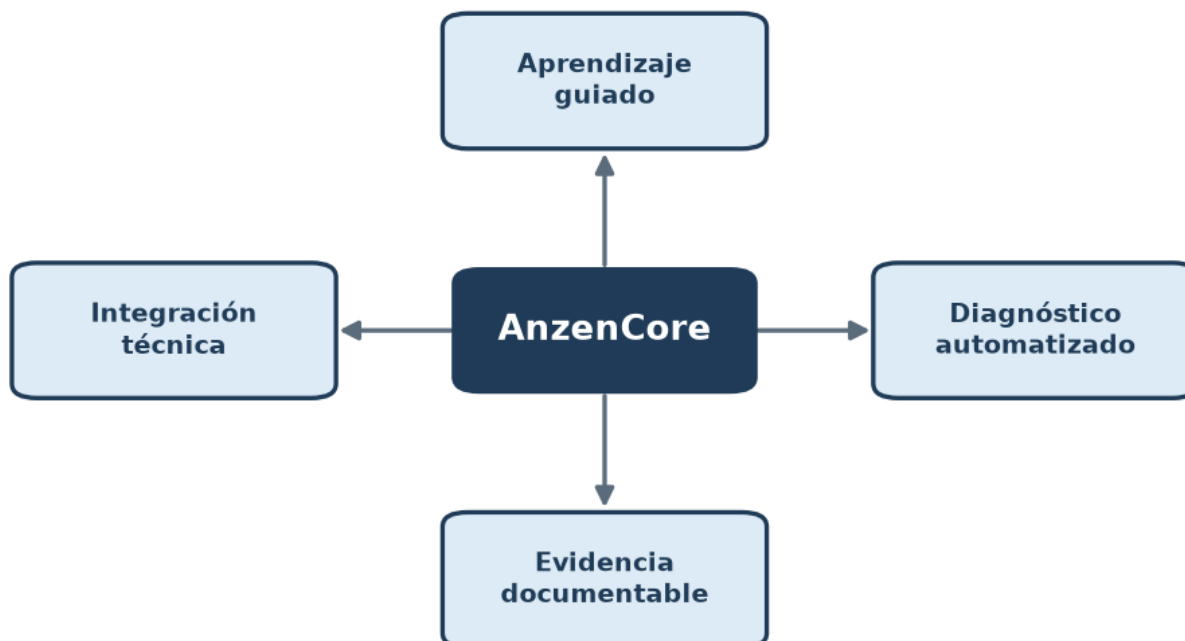


Figura 8. Mapa de valor académico y técnico de AnzenCore. Fuente: elaboración propia.

20.1 Valor diferencial de AnzenCore

Dimensión	Situación tradicional	Aporte de AnzenCore	Evidencia esperada
Aprendizaje	El estudiante revisa el OWASP Mobile Top 10 de forma teórica.	Relaciona teoría con APKs y código reales, diagnóstico y sugerencias.	Capturas de análisis y reportes.
Tiempo	La revisión depende de lectura manual y experiencia del evaluador.	Automatiza diagnóstico preliminar y permite repetir análisis.	Comparación de tiempo antes/después.
Trazabilidad	Correcciones suelen quedar en comentarios sueltos o apuntes.	Genera evidencias, historial o salidas JSON reutilizables.	Historial, reportes o resultados exportados.
Integración	La validación ocurre fuera del flujo de desarrollo.	Permite uso desde Streamlit, API, CLI o VS Code (con análisis de APK 100% local).	Prueba de API, CLI y extensión.
Calidad	El proyecto académico puede no demostrar QA formal.	Incluye pruebas unitarias, API, UI, riesgos y criterios de aceptación.	Plan QA y ejecución de casos.

20.2 Principios rectores de la propuesta

- Claridad pedagógica: cada diagnóstico debe explicar por qué existe una posible vulnerabilidad o code smell.
- Transparencia técnica: toda limitación del motor debe ser reconocida, especialmente en detección basada en patrones/regex.
- Trazabilidad académica: cada resultado debe poder vincularse con un requerimiento, una prueba y una evidencia.
- Arquitectura modular: interfaz, core, API, persistencia, extensión de editor, pruebas y documentación deben poder evolucionar sin acoplamiento excesivo.
- Seguridad responsable: no exponer credenciales, proteger variables de entorno y restringir CORS en despliegue real.
- Evolución incremental: priorizar un producto funcional y demostrable antes que prometer automatizaciones no verificadas.

21. Diagnóstico profundo del proyecto AnzenCore

El repositorio AnzenCore evidencia una estructura modular con archivo principal app.py y carpetas app/api (routes, services, schemas, models, config) y app/dashboard (controllers, views, services, models, config). Esta organización favorece una propuesta sólida porque permite separar vista, controladores, lógica de análisis de vulnerabilidades, persistencia y pruebas.

El README del proyecto declara componentes de valor como Streamlit, PostgreSQL/Supabase, extensión de VS Code publicada en el Marketplace, CLI, detección de vulnerabilidades OWASP Mobile, Docker, Terraform, GitHub Actions, SonarQube, Snyk, Semgrep y pruebas con Pytest/Behave/Mutmut. En el FD06 enriquecido se debe tratar esta información con criterio: no todo lo mencionado debe asumirse como completamente productivo si existen módulos que requieren configuración externa (Supabase) para operar por completo.

21.1 Inventario técnico real del repositorio

Elemento	Ubicación/archivo	Función	Estado documentable
Interfaz web	app.py y app/dashboard/views	Presentar la experiencia Streamlit, pantallas y resultados.	Implementado/parcial según ejecución real.
Motor de análisis de APK	app/dashboard/services/apk_analyzer.py	Descomprimir el APK y detectar vulnerabilidades por patrones.	Implementado con lógica basada en regex.
Motor de code smells	app/api/services/folder_analyzer.py	Analizar código fuente: LOC, complejidad, NOM, NOA y smells.	Implementado.
API	app/api/routes/analizar.py, external_analysis.py	Exponer POST /api/analizar y POST /api/analysis/external/*.	Implementado, con mejora requerida en CORS y errores.
CLI	app/dashboard/services / scripts	Ejecutar análisis por archivo y devolver JSON.	Implementado.
Persistencia	Supabase/PostgreSQL, app/api/routes/auth.py	Soporte para autenticación, usuarios e historial de hallazgos.	Parcial/depende de configuración.
Reportes	app/api/services/report_service.py, report_export_service.py	Generar y exportar reportes de resultados.	Implementado/parcial según dependencias.
Extensión VS Code	vscode-extension/	Analizar workspace, carpeta, archivo o APK desde el editor; APK 100% local.	Implementado y publicado en el Marketplace.
Tests	tests/unit, integration, interface, bdd	Pruebas de núcleo, integración, interfaz y BDD.	Parcial: pruebas deben ampliarse con más casos reales.
Infraestructura	Dockerfile.api, Dockerfile.dashboard, .github, infra/	Contenerización, CI/CD e IaC en Azure Container Apps.	Presente, requiere validación.

21.2 Hallazgos QA relevantes

Código	Hallazgo	Impacto	Recomendación
QA-01	La API permite CORS con allow_origins configurable; en desarrollo puede ser amplio.	Adecuado para demo, riesgoso en producción.	Restringir dominios permitidos antes de despliegue real.
QA-02	Algunas suites de prueba pueden requerir ampliación de casos reales por servicio.	La cobertura real puede variar entre módulos.	Ampliar casos de FolderAnalyzer, ApkAnalyzer y rutas API.
QA-03	La detección de vulnerabilidades se basa en expresiones regulares, no en decompilación completa.	Puede producir falsos positivos o falsos negativos ante ofuscación.	Documentar la limitación y evaluar herramientas de decompilación como mejora futura.
QA-04	El historial y la autenticación dependen de Supabase y variables .env.	El sistema puede fallar si no hay configuración externa.	Validar configuración y documentar modo sin persistencia.
QA-05	Los errores internos deben evitar exponer detalles de excepción al usuario final.	Puede exponer detalles técnicos si no se controla.	Estandarizar errores y registrar trazas solo en servidor.
QA-06	La extensión VS Code requiere validación funcional en distintos entornos Node.	Puede no demostrarse si no se instala y prueba en el equipo del evaluador.	Incluir evidencia de instalación, comandos ejecutados y capturas del panel de resultados.

21.3 Decisiones técnicas que deben mantenerse

La propuesta debe mantener Python como núcleo por su facilidad para procesar texto, aplicar reglas de detección y generar diagnósticos. Streamlit es adecuado para una demostración académica rápida y visual. FastAPI permite demostrar integración HTTP/JSON y documentación OpenAPI. Supabase/PostgreSQL ofrece una base de persistencia viable para historial y autenticación, pero debe usarse sin exponer credenciales. Docker, GitHub Actions y Terraform fortalecen la dimensión profesional siempre que se documenten como capacidades verificables y no solo decorativas. La extensión de VS Code, al ejecutar el análisis de APK 100% en local, refuerza la propuesta de valor sin depender de infraestructura desplegada.

22. Análisis estratégico y propuesta de valor ampliada

22.1 Población objetivo segmentada

Segmento	Necesidad principal	Dolor actual	Respuesta de AnzenCore
Estudiantes iniciales de seguridad móvil	Comprender el OWASP Mobile Top 10 con ejemplos.	Confunden tipos de vulnerabilidad y su severidad real.	Diagnóstico guiado por severidad, CWE y OWASP Mobile.
Estudiantes con proyectos móviles	Validar la seguridad de su APK antes de sustentación.	No tienen evidencia técnica de vulnerabilidades.	Reporte y recomendaciones trazables.
Docentes	Revisar muchos proyectos de forma preliminar.	La revisión repetitiva consume tiempo.	Herramienta de apoyo, no reemplazo del juicio docente.
Desarrolladores principiantes	Detectar vulnerabilidades comunes en apps Android.	Usan patrones inseguros (HTTP plano, secretos hardcoded).	Recomendaciones sobre remediación y código seguro.
Evaluador académico	Ver coherencia entre código, documentos y pruebas.	Proyectos suelen carecer de QA y trazabilidad.	FD06 integra alcance, presupuesto, riesgos y QA.

22.2 Propuesta de valor resumida

AnzenCore ofrece una propuesta de valor basada en cuatro elementos: aprendizaje guiado, diagnóstico automatizado, evidencia documentable e integración técnica. Esto significa que el estudiante no solo obtiene un resultado, sino una explicación que puede usar para corregir su código o APK y sustentar su decisión frente al docente.

Eje de valor	Descripción	Indicador de éxito
Aprendizaje	El sistema explica hallazgos de seguridad en lenguaje comprensible.	80% de usuarios piloto entiende la causa del diagnóstico.
Velocidad	El análisis de un APK se realiza en segundos, incluso de forma local en VS Code.	Reducción de al menos 50% del tiempo preliminar de revisión.
Calidad	El resultado se evalúa con pruebas y criterios de aceptación.	90% de pruebas críticas aprobadas.
Trazabilidad	Cada análisis puede registrarse, exportarse o evidenciarse.	Evidencia disponible por cada caso de prueba.
Escalabilidad académica	El sistema puede ser usado por múltiples estudiantes en prácticas.	Reutilización en más de una actividad académica.

22.3 Alineamiento con objetivos académicos

- Apoya el logro de competencias en calidad y seguridad de software.
- Facilita la identificación de vulnerabilidades del OWASP Mobile Top 10 en aplicaciones Android.

- Promueve buenas prácticas de desarrollo seguro antes de publicar una aplicación.
- Relaciona teoría de seguridad móvil con código Java/Kotlin y estructura real de APK.
- Permite convertir la revisión de vulnerabilidades en un proceso repetible y documentado.
- Refuerza habilidades de programación, pruebas, API, documentación y DevOps.

23. EDT/WBS, backlog e historias de usuario

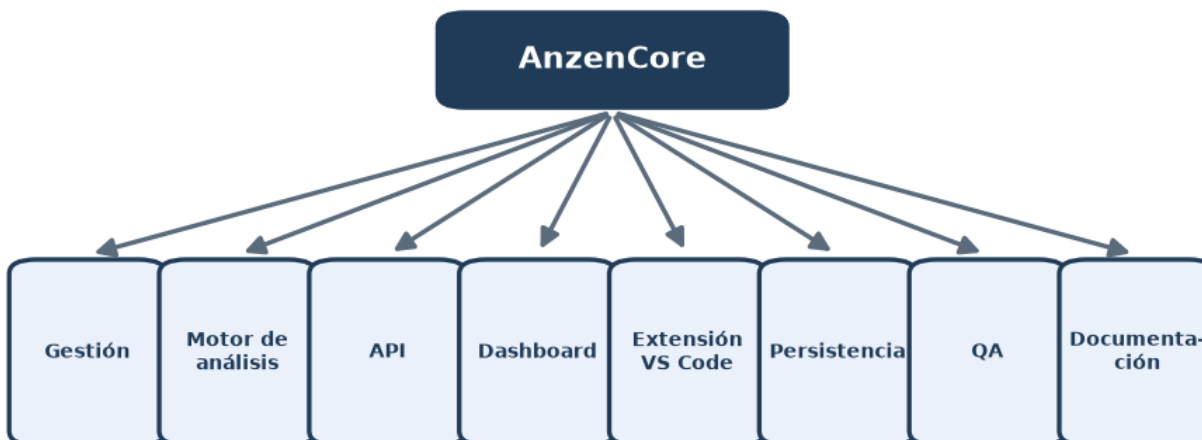


Figura 9. Estructura de desglose del trabajo para AnzenCore. Fuente: elaboración propia.

23.1 Backlog de producto priorizado

ID	Épica	Historia de usuario	Prioridad	Entrega
BP-01	Ingreso de APK	Como estudiante quiero subir un APK para iniciar el análisis de vulnerabilidades.	Alta	MVP
BP-02	Motor de análisis de APK	Como sistema quiero descomprimir el APK y detectar hallazgos de seguridad.	Alta	MVP
BP-03	Detección de secretos	Como usuario quiero saber si existen secretos hardcodeados en el código.	Alta	MVP
BP-04	Detección de HTTP inseguro	Como usuario quiero detectar comunicación sin cifrar en la app.	Alta	MVP
BP-05	Detección de criptografía débil	Como usuario quiero identificar el uso de algoritmos criptográficos obsoletos.	Alta	MVP
BP-06	Recomendaciones	Como estudiante quiero recibir recomendaciones concretas de remediación.	Alta	MVP
BP-07	Reporte visual	Como usuario quiero ver resultados ordenados por severidad.	Alta	MVP
BP-08	API	Como integrador quiero consumir el motor desde POST /api/analizar.	Alta	MVP
BP-09	Historial	Como usuario quiero guardar análisis para comparar avances.	Media	Incremento 2
BP-10	Exportación	Como usuario quiero descargar un reporte con los hallazgos.	Media	Incremento 2
BP-11	VS Code	Como desarrollador quiero analizar código y APKs desde mi editor, incluso sin conexión.	Alta	MVP
BP-12	QA automatizado	Como equipo quiero pruebas reales que respalden la calidad.	Alta	MVP
BP-13	Seguridad	Como responsable quiero evitar exposición de secretos y CORS abierto.	Alta	MVP
BP-14	Documentación	Como evaluador quiero manuales, evidencias y trazabilidad.	Alta	MVP

23.2 Historias de usuario detalladas con criterios de aceptación

ID	Nombre	Criterio de aceptación	Condición	Prioridad
HU-01	Analizar APK válido	Dado un APK válido, cuando el usuario solicita análisis, entonces el sistema muestra hallazgos, severidad y recomendaciones.	APK válido y no corrupto.	Alta
HU-02	Controlar entrada vacía	Dado un archivo vacío, cuando se llama a la API, entonces devuelve un error 400 controlado.	No debe lanzar traceback al usuario.	Alta
HU-03	Detectar secreto hardcodeado	Dado un código con un patrón de API key o token, entonces el diagnóstico reporta hardcoded_secret con CWE-798.	Evidencia enmascarada en el reporte.	Alta
HU-04	Detectar HTTP inseguro	Dado un endpoint http:// en el código, entonces reporta insecure_communication con CWE-319 y OWASP M5.	Ignora URLs de licencias/esquemas conocidos.	Alta
HU-05	Detectar criptografía débil	Dado el uso de MD5, SHA-1, DES o AES/ECB, entonces reporta weak_crypto con CWE-327.	Explica el algoritmo débil detectado.	Alta
HU-06	Detectar WebView inseguro	Dado setJavaScriptEnabled(true), entonces marca insecure_webview con CWE-79.	Sugiere restringir el origen del contenido.	Media

HU-07	Generar reporte JSON	Dado un análisis por API, entonces devuelve resumen, severidad máxima y lista de vulnerabilidades.	Contrato JSON estable.	Alta
HU-08	Guardar historial	Dado un usuario configurado, entonces se registra el análisis en PostgreSQL/Supabase.	Aplica solo si variables .env están configuradas.	Media
HU-09	Exportar reporte	Dado un diagnóstico, entonces el usuario puede exportar los hallazgos en un reporte descargable.	Debe reflejar los mismos datos que el dashboard.	Media
HU-10	Analizar desde VS Code	Dado un archivo .apk en el Explorer, entonces la extensión muestra un panel con las vulnerabilidades encontradas.	Funciona sin conexión al servidor.	Alta
HU-11	Analizar carpeta desde VS Code	Dado un workspace abierto, cuando se ejecuta "Analizar Workspace", entonces se muestran code smells por archivo.	Requiere la API disponible.	Alta
HU-12	Detectar APK automáticamente	Dado un proyecto con archivos .apk, cuando se analiza el workspace, entonces la extensión ofrece analizarlos.	Detección mediante búsqueda de archivos.	Media
HU-13	Controlar archivo corrupto	Dado un archivo que no es un ZIP/APK válido, entonces el sistema muestra un error comprensible.	Sin caída total de la interfaz o extensión.	Alta
HU-14	Pruebas automatizadas	Dado el repositorio, cuando se ejecuta pytest, entonces las pruebas críticas validan los servicios de análisis.	Cobertura sobre ApkAnalyzer y FolderAnalyzer.	Alta
HU-15	Sustentación docente	Dado el FD06 final, entonces existe coherencia entre problema, objetivos, solución, costos, riesgos y QA.	Checklist jurado completo.	Alta

24. Arquitectura técnica, ciclo de datos, API y seguridad

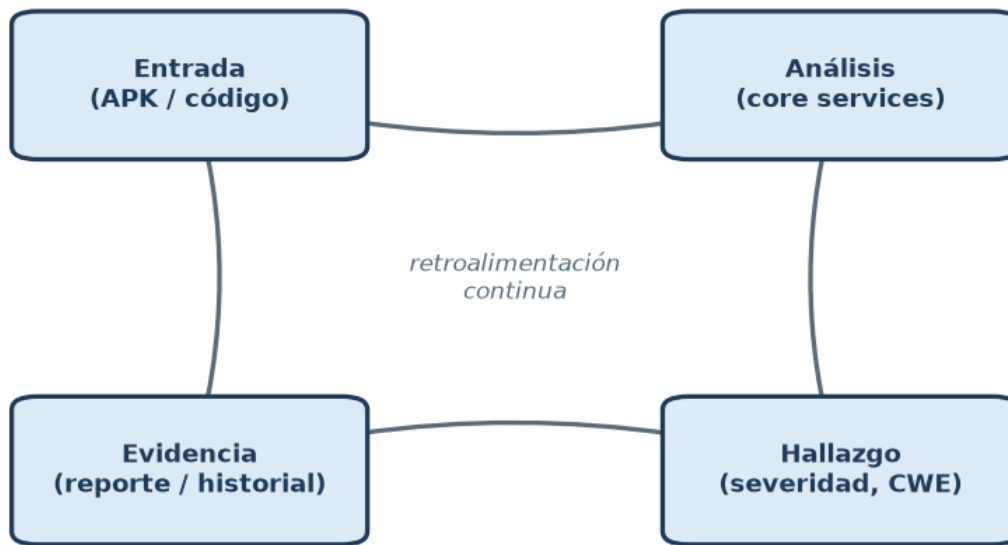


Figura 10. Ciclo de datos y evidencia del flujo de análisis. Fuente: elaboración propia.

24.1 Arquitectura lógica enriquecida

La arquitectura se interpreta como una variante MVC extendida: las vistas corresponden a Streamlit y potencialmente VS Code; los controladores se ubican en `app/dashboard/controllers` y las rutas de `app/api/routes`; el modelo lógico vive en `app/api/services` y `app/dashboard/services`; la persistencia y autenticación residen en Supabase/PostgreSQL; la API externa se expone con FastAPI. Esta separación facilita pruebas, mantenimiento y ampliación futura.

Capa	Responsabilidad	Módulos relacionados	Riesgo principal	Mitigación
Presentación	Capturar APK/código, mostrar diagnóstico y evidencias.	<code>app.py</code> , <code>app/dashboard/views</code>	Interfaz poco clara para principiantes.	Mensajes pedagógicos, ejemplos y severidad visible.
Controladores	Coordinar flujo entre UI, core y datos.	<code>app/dashboard/controllers</code> , <code>app/api/routes</code>	Acoplamiento con lógica de negocio.	Mantener controladores delgados.
Core	Analizar APK/código y generar hallazgos.	<code>apk_analyzer.py</code> , <code>folder_analyzer.py</code> , <code>vulnerability_scanner.py</code>	Falsos positivos/negativos por regex.	Documentar limitaciones y permitir revisión manual.
API	Exponer servicio HTTP/JSON.	<code>app/api/routes</code> , <code>app/api/services</code>	CORS abierto o errores expuestos.	Restringir CORS y estandarizar errores.
Datos	Guardar usuarios, historial y resultados.	Supabase/PostgreSQL	Configuración externa faltante.	Variables .env y modo local sin persistencia.
DevOps	Construir, probar y desplegar.	<code>Dockerfile.api</code> , <code>Dockerfile.dashboard</code> , <code>.github</code> , <code>infra/</code>	Scripts no validados.	Pipeline con pruebas reales.

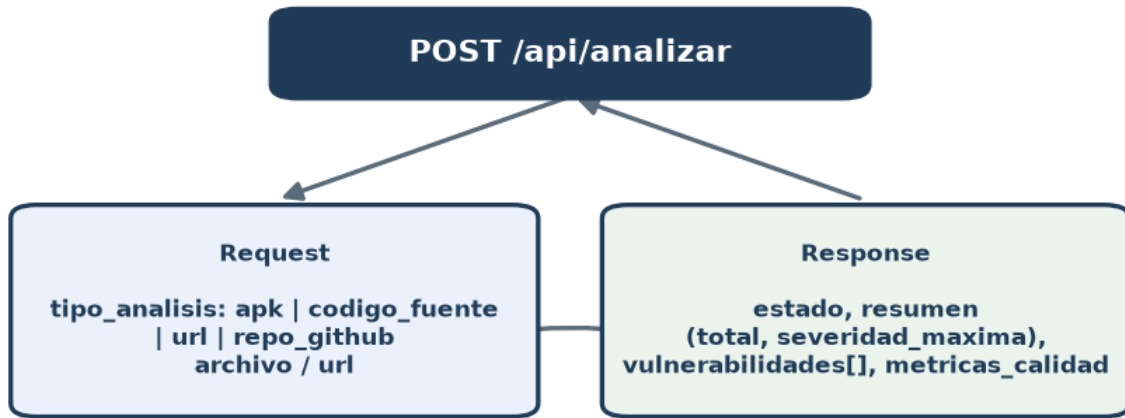


Figura 11. Contrato conceptual de la API de análisis de vulnerabilidades. Fuente: elaboración propia.

24.2 Especificación ampliada del endpoint principal

Elemento	Detalle
Método y ruta	POST /api/analizar
Propósito	Recibir un APK, código fuente, URL o repositorio y devolver un diagnóstico de vulnerabilidades y calidad.
Entrada esperada	tipo_analisis (apk codigo_fuente url repo_github) + archivo o url según el tipo.
Salida exitosa	estado, resumen (total_vulnerabilidades, severidad_maxima, conteo_por_severidad), vulnerabilidades, metricas_calidad.
Error por entrada vacía	HTTP 400 con detail="El archivo esta vacio." o similar.
Error por tipo inválido	HTTP 400 indicando los valores válidos de tipo_analisis.
Error interno	Debe devolver mensaje controlado sin exponer detalles sensibles en versión final.
Prueba recomendada	Postman/curl con APK válido, APK corrupto, código fuente con vulnerabilidades, URL y JSON inválido.
Mejora prioritaria	Agregar autenticación opcional, límite de tamaño de payload y CORS restringido para producción.

24.3 Seguridad y privacidad

Riesgo	Control mínimo	Control recomendado	Evidencia
Credenciales expuestas	Uso de .env y .env.example.	Secret scanning en CI y rotación de claves.	Archivo .env no versionado.
CORS abierto	Permitido solo para demo local.	Lista blanca de orígenes confiables.	Configuración documentada en app/api/main.py.
Datos de análisis privados	No compartir historial entre usuarios.	Row Level Security en Supabase/PostgreSQL.	Prueba de acceso cruzado.
Errores internos	Mensaje controlado al usuario.	Logs separados del output público.	Captura de error controlado.
Uso abusivo de API	Validar tamaño de entrada (APK/archivos).	Rate limiting y timeout.	Prueba con payload grande.

24.4 Plan DevOps e infraestructura

La presencia de Dockerfiles independientes, GitHub Actions y Terraform permite proponer un flujo DevOps académico. Sin embargo, el FD06 debe exigir evidencia: construcción de imagen, ejecución local, pruebas automatizadas y documentación de variables. No se debe afirmar despliegue productivo si solo existe configuración inicial.

Fase	Actividad	Herramienta	Criterio de aceptación
Build	Construir entorno reproducible.	Dockerfile.api, Dockerfile.dashboard	Imagen construye sin errores y ejecuta la app.
Test	Ejecutar pruebas automáticas.	pytest, behave, Postman	Pruebas críticas con asserts reales.
Security	Detectar secretos y dependencias inseguras.	Snyk, Semgrep, secret scan	Reporte sin secretos expuestos.
Deploy demo	Ejecutar Streamlit y API en entorno local o nube.	Streamlit, uvicorn, Docker, Azure Container Apps	URLs accesibles y evidencias.
IaC	Documentar infraestructura como código.	Terraform (infra/)	Plan documentado, sin credenciales en variables.

25. Plan QA extendido y criterios de aceptación

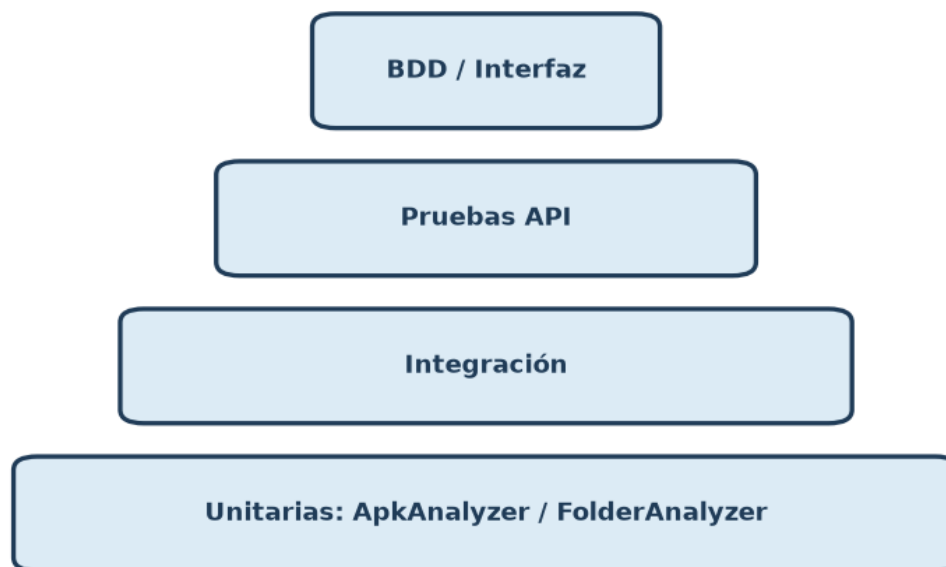


Figura 12. Estrategia de pruebas por niveles para AnzenCore. Fuente: elaboración propia.

25.1 Objetivo del plan QA

El plan de calidad tiene como objetivo demostrar que AnzenCore no solo funciona en casos favorables, sino que responde de forma controlada ante errores, entradas ambiguas, APKs corruptos y configuraciones incompletas. El proyecto debe evolucionar y mantener pruebas reales con entradas representativas (APKs y código con vulnerabilidades conocidas).

25.2 Matriz de pruebas funcionales principales

ID	Escenario	Entrada	Resultado esperado	Prioridad
TC-01	Entrada vacía	archivo vacío en tipo analisis=apk	Error 400 controlado	Alta
TC-02	Archivo no es un ZIP/APK	archivo .txt renombrado a .apk	Mensaje: archivo APK inválido o corrupto	Alta
TC-03	APK sin vulnerabilidades evidentes	APK de ejemplo simple (assets/app-debug.apk)	Sin hallazgos críticos, severidad Info/Medio	Alta
TC-04	APK sin AndroidManifest.xml	APK manipulado sin manifest	Hallazgo "manifest" severidad Alta	Alta
TC-05	Secreto hardcodeado	cadena tipo apikey: sk live XXXXXXXXXXXXXXXXX	Hallazgo hardcoded_secret, CWE-798	Alta
TC-06	HTTP inseguro	URL http://ejemplo.com/api en el código	Hallazgo insecure_communication, CWE-319	Alta
TC-07	Criptografía débil	MessageDigest.getInstance("MD5")	Hallazgo weak_crypto, CWE-327	Alta
TC-08	WebView inseguro	setJavaScriptEnabled(true)	Hallazgo insecure_webview, CWE-79	Media
TC-09	API JSON válido	POST /api/analizar con APK correcto	estado=ok, resumen y vulnerabilidades	Alta
TC-10	API JSON inválido	payload sin campo requerido	Error de validación 400	Alta
TC-11	Análisis de código fuente	carpeta con secretos e IP hardcodeada	code_smells.files con hallazgos por archivo	Alta
TC-12	Historial sin Supabase	sin variables .env configuradas	No debe bloquear el análisis local	Media
TC-13	Extensión VS Code: APK local	clic derecho en .apk en el Explorer	Panel de vulnerabilidades sin llamada de red	Alta
TC-14	Carga de archivo grande	APK de varios MB (~12MB)	Tiempo de análisis local menor a 2 segundos	Media
TC-15	Regresión core	suite pytest	Todas las pruebas críticas pasan	Alta

25.3 Criterios de aceptación globales

- El sistema debe analizar al menos un APK y un código fuente válidos desde interfaz web, API y CLI.
- El endpoint POST /api/analizar debe responder con JSON estable para casos válidos e inválidos.
- El diagnóstico debe separar hallazgos por tipo de vulnerabilidad y severidad.
- Las recomendaciones deben ser comprensibles y accionables para un estudiante.
- Los errores no deben mostrar trazas internas al usuario final en modo de entrega.
- La documentación debe explicar limitaciones del enfoque basado en patrones/regex.
- Los tests deben cubrir casos reales de vulnerabilidades antes de la sustentación final.
- El documento FD06 debe mantener consistencia con FD03, FD05, repositorio y evidencias.

26. Gobierno, monitoreo, comunicación y adopción



Figura 13. Modelo de gobierno académico para AnzenCore. Fuente: elaboración propia.

26.1 Matriz RACI extendida

Actividad	Estudiante desarrollador	Docente	QA/Revisor	Usuario piloto	Evidencia
Definir alcance	R	A	C	C	Acta o sección de alcance
Diseñar arquitectura	R	C	C	I	Diagrama y explicación
Implementar motor de análisis	R/A	I	C	I	Código app/api/services, app/dashboard/services
Implementar API y extensión VS Code	R/A	I	C	I	app/api/routes, Postman y .vsix publicado
Pruebas funcionales	R	I	A	C	Matriz QA
Validación académica	C	A	R	C	Rúbrica y observaciones
Documentación FD06	R/A	C	C	I	Documento final
Sustentación	R	A	C	I	Presentación y demo

26.2 Plan de monitoreo y evaluación enriquecido



Figura 14. Scorecard de monitoreo académico para AnzenCore. Fuente: elaboración propia.

Indicador	Fórmula	Meta	Frecuencia	Responsable	Evidencia
Avance de requerimientos	RF implementados / RF planificados	>= 80% MVP	Semanal	Desarrollador	Backlog actualizado
Pruebas aprobadas	Pruebas OK / pruebas ejecutadas	>= 90% críticas	Por sprint	QA	Reporte pytest/Postman
Errores críticos abiertos	Cantidad de errores severidad alta	0 al cierre	Semanal	QA	Issue tracker
Tiempo de análisis	Promedio por APK/código	< 2 s en análisis local de VS Code	Sprint 6+	Desarrollador	Log de ejecución
Cobertura documental	Secciones completas / secciones requeridas	100% FD06	Final	Documentador	DOCX final
Satisfacción piloto	Promedio encuesta 1-5	>= 4	Cierre	Docente/usuario	Formulario o acta
Trazabilidad	RF con prueba y evidencia / RF total	>= 85%	Final	QA	Matriz trazabilidad
Seguridad básica	Controles cumplidos / controles definidos	>= 90%	Final	Dev/QA	Checklist seguridad

26.3 Plan de comunicación

Audiencia	Mensaje clave	Canal	Frecuencia	Responsable
Docente	Estado, evidencias, riesgos y decisiones de alcance.	Documento, correo, sustentación	Por entrega	Estudiante
Usuario estudiante	Cómo subir un APK/código, interpretar resultados y corregir.	Manual y demo	Sprint final	Estudiante
QA/Revisor	Casos de prueba, defectos y evidencias.	Matriz QA	Semanal	QA
Equipo técnico	Cambios en core, API, persistencia y extensión VS Code.	README, issues	Por sprint	Desarrollador
Evaluador académico	Valor, viabilidad, presupuesto y alcance.	FD06 y exposición	Entrega final	Responsable proyecto

26.4 Plan de capacitación y adopción

La adopción de AnzenCore se organiza en una demostración guiada. Primero se explica el problema académico de seguridad móvil; luego se muestra un APK con vulnerabilidades, se ejecuta el diagnóstico, se interpreta el resultado y se corrige el hallazgo. Finalmente se enseña el uso de la API, CLI o extensión de VS Code para demostrar integración técnica.

Sesión	Tema	Duración	Resultado esperado
S1	Introducción a AnzenCore y OWASP Mobile Top 10	30 min	Usuario entiende propósito y límites.
S2	Uso del dashboard y lectura de resultados	45 min	Usuario ejecuta un análisis completo.
S3	Casos de vulnerabilidad y corrección	45 min	Usuario interpreta hallazgos y recomendaciones.
S4	API, CLI y extensión de VS Code para integración técnica	30 min	Usuario prueba el endpoint, el CLI o analiza un APK local desde VS Code.
S5	Sustentación y evidencias	30 min	Usuario sabe presentar resultados.

27. Presupuesto enriquecido y evaluación financiera académica

27.1 Supuestos presupuestales

El presupuesto de AnzenCore se plantea como referencial académico. No representa una propuesta comercial oficial, pero permite demostrar planificación realista. Se consideran costos de desarrollo, pruebas, documentación, diseño, despliegue y soporte. Como el proyecto usa tecnologías mayormente libres, el mayor costo está en horas de trabajo técnico y documentación.

Supuesto	Valor considerado	Justificación
Duración	4 meses	Periodo académico razonable para análisis, construcción, QA y entrega.
Equipo base	1 responsable principal con apoyo QA/documentación	Adecuado para proyecto universitario de 2 integrantes.
Infraestructura	Entorno local y servicios gratuitos o de bajo costo	Streamlit/FastAPI/Supabase/Azure pueden operar en demo.
Licencias	Preferentemente software libre	Python, FastAPI, Streamlit, PostgreSQL y Node.js son accesibles.
Contingencia	10% a 15%	Cubre ajustes, pruebas adicionales y documentación final.

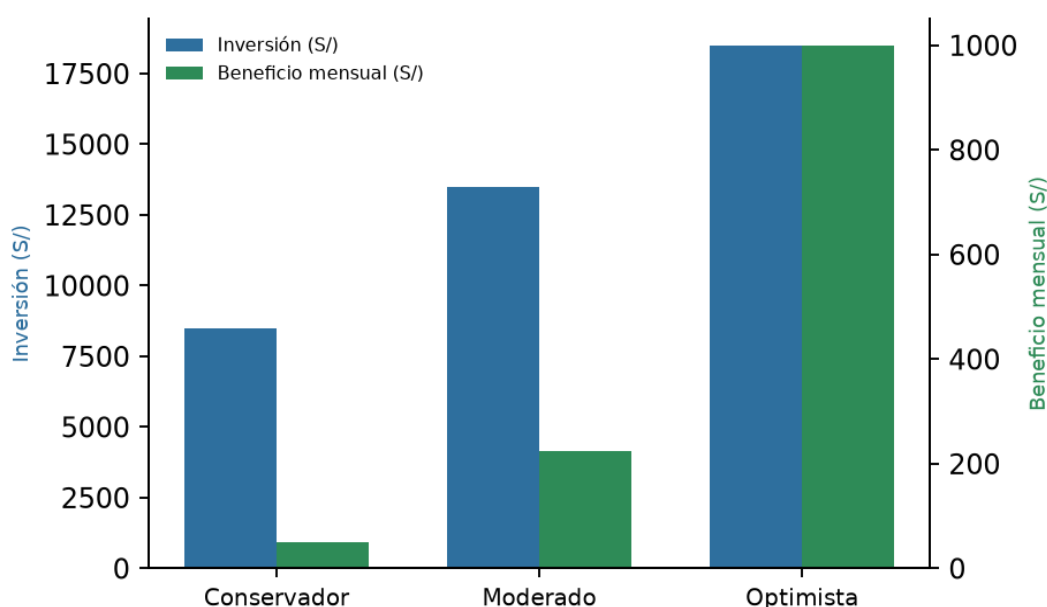


Figura 15. Indicadores de valor usados para justificar la inversión académica. Fuente: elaboración propia.

27.2 Presupuesto por escenarios

Categoría	Mínimo S/	Recomendado S/	Extendido S/	Observación
Análisis y gestión	800	1,200	1,600	Alcance, cronograma, riesgos y seguimiento.
Desarrollo motor de análisis Python	2,200	3,200	4,200	ApkAnalyzer, FolderAnalyzer y detectores de vulnerabilidad.
Dashboard Streamlit	1,000	1,600	2,200	Vistas, UX, mensajes y evidencias.
API FastAPI	900	1,400	2,000	Endpoints, validaciones, errores y docs.
Persistencia Supabase/PostgreSQL	700	1,100	1,600	Historial, autenticación y configuración.
Extensión VS Code	600	1,000	1,600	Comandos, panel de resultados y publicación en el Marketplace.
QA y pruebas	1,000	1,800	2,600	Unit, API, UI, BDD y regresión.
Documentación	1,200	2,000	2,800	FD03, FD05, FD06, manuales y anexos.
Despliegue/DevOps	500	900	1,400	Docker, Terraform, CI/CD, configuración.
Capacitación/sustentación	500	800	1,200	Demo, guion, entrenamiento.
Contingencia	900	1,500	2,300	Ajustes y mejoras finales.
TOTAL	10,300	16,500	23,500	Montos referenciales académicos.

27.3 Retorno académico esperado

El retorno de AnzenCore no debe medirse únicamente como ingreso monetario, sino como valor académico: ahorro de tiempo de revisión de seguridad, mejora de comprensión, reutilización en prácticas, reducción de vulnerabilidades no detectadas y fortalecimiento de evidencias para sustentación. Aun así, puede estimarse un ahorro referencial de horas si el sistema reduce revisiones manuales repetitivas.

Escenario	Uso mensual estimado	Ahorro por análisis	Valor hora S/	Beneficio mensual S/	Retorno estimado
-----------	----------------------	---------------------	---------------	----------------------	------------------

AnzenCore - FD06 / Propuesta de Proyecto

Conservador	20 análisis	10 min	15	50	Valor principalmente académico.
Moderado	60 análisis	15 min	15	225	Útil para cursos y prácticas.
Optimista	150 análisis	20 min	20	1,000	Viable como herramienta institucional piloto.

27.4 Evaluación beneficio/costo referencial

Indicador	Cálculo referencial	Interpretación
Inversión recomendada	S/ 16,500	Incluye desarrollo, QA, documentación y contingencia.
Beneficio académico anual moderado	S/ 2,700 en ahorro de tiempo + valor formativo	El retorno monetario no captura todo el valor educativo y de seguridad.
Beneficio/costo directo	0.16 en primer año solo por ahorro de tiempo	No es un producto comercial puro; es inversión educativa.
Beneficio/costo ampliado	Mayor a 1 si se reutiliza en cursos, prácticas y portafolio	El valor aumenta con reutilización institucional.
Periodo de maduración	1 a 2 ciclos académicos	Tiempo razonable para mejora y adopción.

28. Riesgos, mitigaciones y trazabilidad ampliada

28.1 Matriz de riesgos extendida

ID	Riesgo	Prob.	Impacto	Nivel	Mitigación
R-01	El motor no interpreta todas las variantes de código u ofuscación del APK.	Media	Alta	Alto	Limitar alcance de detección y documentar casos soportados.
R-02	Las vulnerabilidades basadas en regex generan falsos positivos o negativos.	Alta	Alta	Crítico	Documentar limitaciones y priorizar revisión manual en casos ambiguos.
R-03	Cobertura de pruebas insuficiente en algunos módulos.	Media	Media	Alto	Ampliar pruebas unitarias reales sobre analizadores y rutas API.
R-04	Supabase no está configurado en demo.	Media	Media	Medio	Preparar modo local sin persistencia.
R-05	La API expone CORS abierto.	Media	Alta	Alto	Restringir allow_origins y documentar modo demo.
R-06	Credenciales en archivos o repositorio.	Baja	Alta	Alto	Usar .env, .gitignore y secret scanning.
R-07	El documento promete funcionalidades no verificadas.	Media	Alta	Alto	Clasificar implementado/partial/planificado.
R-08	El usuario no entiende el diagnóstico de seguridad.	Media	Media	Medio	Mensajes pedagógicos y ejemplos antes/después.
R-09	La extensión VS Code no se puede demostrar en el equipo del evaluador.	Media	Media	Medio	Preparar evidencia en video o capturas del panel de resultados.
R-10	Sobrecarga de alcance en 4 meses.	Media	Alta	Alto	Priorizar MVP: motor de APK + code smells + API + extensión VS Code.
R-11	Errores internos se muestran al usuario.	Media	Media	Medio	Estandarizar mensajes y logs privados.
R-12	Falta de evidencia visual final.	Media	Media	Medio	Reservar sprint final para capturas y anexos.

28.2 Trazabilidad problema-objetivo-requerimiento-prueba

ID	Problema asociado	Objetivo	RF	Prueba/evidencia	Prioridad
TR-01	Análisis de seguridad manual lento	OE1/OE5	RF-01/RF-12	TC-01, TC-03	Alta
TR-02	Desconocimiento de secretos hardcoded	OE2	RF-03	TC-05	Alta
TR-03	Dificultad con HTTP inseguro y criptografía débil	OE2/OE3	RF-04/RF-05	TC-06, TC-07	Alta
TR-04	Dificultad con WebView y Random inseguros	OE3	RF-06/RF-07	TC-08	Alta
TR-05	Falta de evidencia	OE6/OE10	RF-14/RF-20	TC-09, checklist	Alta
TR-06	Necesidad de integración técnica	OE7	RF-10/RF-11	TC-09, TC-11	Alta
TR-07	Riesgos de calidad	OE9	RF-17/RF-18	TC-15	Media
TR-08	Persistencia de resultados	OE8	RF-15	TC-12	Media